

JAVA DOCS

JVM, JRE, and JDK

Java Runtime Internals, Memory Model & Garbage Collection

Java Agentic AI — Knowledge Base Reference

1. The Three Layers

Component	Contains	Purpose
JVM (Java Virtual Machine)	Class loader, runtime data areas, execution engine	Executes bytecode; the abstract 'computer' that makes the JVM work
JRE (Java Runtime Environment)	JVM + core class libraries (java.lang, java.util, ...)	Everything needed to RUN a compiled Java application
JDK (Java Development Kit)	JRE + javac, javadoc, jar, jdb, jshell, etc.	Everything needed to DEVELOP and build Java applications

Note: Since Java 11, Oracle no longer ships a separate standalone JRE download — the JDK is the standard distribution, and tools like `jlink` can produce minimal custom runtime images.

2. JVM Architecture

The JVM has three major subsystems: the **Class Loader Subsystem**, **Runtime Data Areas** (memory), and the **Execution Engine**.

2.1 Class Loader Subsystem

- **Loading:** reads .class bytecode and constructs a Class object in memory. Delegation model: Bootstrap ClassLoader (core JDK classes, written in native code) -> Platform/Extension ClassLoader -> Application ClassLoader (classpath). Each delegates to its parent first before loading itself.
- **Linking:** Verification (bytecode is structurally valid and safe) -> Preparation (static fields allocated with default values) -> Resolution (symbolic references resolved to direct references).
- **Initialization:** static initializers and static field assignments run, in source order.

2.2 Runtime Data Areas (Memory)

Area	Shared?	Contains
Heap	Shared across all threads	All objects and arrays; managed by the garbage collector
Method Area / Metaspace	Shared	Class metadata, static variables, constant pool, method bytecode
Stack	Per-thread	Stack frames — local variables, operand stack, per method call
PC Register	Per-thread	Address of the currently executing JVM instruction
Native Method Stack	Per-thread	Support for native (JNI) method calls

Note: Metaspace (Java 8+) replaced PermGen and lives in native (off-heap) memory by default, growing dynamically rather than throwing `java.lang.OutOfMemoryError: PermGen space` with a fixed small cap.

2.3 Execution Engine

- **Interpreter:** reads and executes bytecode instruction-by-instruction — simple but slow for hot code paths.
- **JIT (Just-In-Time) Compiler:** identifies 'hot spots' (frequently executed code) and compiles them directly to optimized native machine code at runtime, dramatically improving performance for long-running code.

- **Garbage Collector:** automatically reclaims heap memory occupied by unreachable objects.

3. Heap Structure & Generational GC

The heap is divided by generational hypothesis: most objects die young, so segregating young and old objects lets the GC focus effort where it pays off.

Region	Description
Young Generation (Eden + 2 Survivor spaces)	New objects allocated here; Minor GC runs frequently and is fast
Old / Tenured Generation	Objects that survive several Minor GC cycles get 'promoted' here; Major/Full GC

```
// Common JVM memory flags
java -Xms256m -Xmx2g -XX:+UseG1GC MyApp
// -Xms initial heap size
// -Xmx maximum heap size
// -XX:+UseG1GC selects the G1 garbage collector
```

4. Garbage Collectors

Collector	Characteristics	Best for
Serial GC	Single-threaded, stop-the-world	Small applications, single-core environments
Parallel GC	Multi-threaded young+old, stop-the-world	Throughput-focused batch workloads
G1 GC (default since Java 9)	Region-based, mostly concurrent, predictable pause targets	General-purpose, low-latency needs
ZGC / Shenandoah	Concurrent, sub-millisecond pauses even on huge heaps	Very large heaps, ultra-low-latency services

5. How Garbage Collection Decides Reachability

An object is eligible for collection when it's unreachable from any **GC root** (active thread stacks, static fields, JNI references). The classic mark-sweep(-compact) algorithm: **mark** all reachable objects starting from GC roots; **sweep** reclaims unmarked (garbage) objects; **compact** optionally defragments the remaining live objects to avoid heap fragmentation.

```
Object obj = new Object(); // reachable via local variable 'obj'
obj = null;                // now unreachable (assuming no other reference exists)
                           // eligible for GC on the next collection cycle
System.gc();               // only a HINT to the JVM - not guaranteed to run
immediately
```

6. Memory Leaks in Java (Yes, They're Possible)

Even with automatic GC, an object can be unintentionally kept reachable, preventing collection:

- Static collections that grow forever and are never cleared.
- Unclosed resources (streams, connections) holding native/heap memory.

- Listener/callback registrations that are never removed.
- Inner (non-static) classes holding an implicit reference to a long-lived outer instance.
- ThreadLocal values not removed, especially in pooled-thread environments (e.g. app servers).

7. Quick Interview Recap

- Difference between JDK, JRE, JVM? JVM executes bytecode; JRE = JVM + libraries to run apps; JDK = JRE + development tools to build apps.
- What replaced PermGen? Metaspace, in native memory, growing dynamically (Java 8+).
- What is JIT compilation? Runtime compilation of hot bytecode paths into optimized native machine code for performance.
- Why generational GC? Because empirically most objects die young — segregating young/old objects lets frequent, cheap Minor GCs handle the majority of garbage efficiently.
- Does System.gc() guarantee collection? No — it's only a suggestion to the JVM; the actual GC algorithm decides if/when to run.